

AAES-OS Code Breakdown

Adaptive AI Execution System — Operating System
Technical Implementation Reference | Version 1.0 | June 2026

Author:	Jon Halstead
Date:	Tuesday, June 16, 2026
Stack:	Python / TypeScript / Node.js / PostgreSQL / Redis / Docker
Classification:	Internal Technical Reference — Confidential
Status:	Active Development — v1.0 Baseline

Table of Contents

- 1. Repository Structure
- 2. Layer-by-Layer Module Breakdown
 - 2.0 Layer 0 — Prompt Compiler Layer (PCL)
 - 2.1 Layer 1 — AAES Alpha: Creative Organism
 - 2.2 Layer 2 — AAES Beta: Governance Intelligence
 - 2.3 Layer 3 — AAES Gamma: Adaptive Intelligence
 - 2.4 Layer 4 — AAES Delta: Research + Simulation
 - 2.5 Layer 5 — AAES Epsilon: Action Layer
 - 2.6 Layer 6 — AAES Zeta: Stewardship
 - 2.7 Layer 7 — AAES Omega: Coordination
 - 2.8 Layer 8 — Genesis Protocol
- 3. API Surface Definition
- 4. Database Schema
- 5. Technology Stack Matrix
- 6. Environment Variables and Configuration
- 7. Deployment Architecture

8. Testing Strategy

9. Build Phase Implementation Checklist

10. Appendices

1. Repository Structure

The following is the canonical directory and file tree for the AAES-OS monorepo. All core engine layers, venture modules, API surface, database layer, frontend, and infrastructure are housed under a single repository root to enable unified versioning, shared tooling, and coherent dependency management.

```
aaes-os/ |— README.md |— package.json |— docker-compose.yml
|— .env.example |— core/ # Core AAES-OS
engine (Layers 0-8) | | |— layer0_prompt_engine/ # PCL –
Prompt Compiler Layer | | |— pcl.py # Main compiler
entry point | | |— shorthand_codes.py # Code registry ([FPS],
[HUD],[ARCH],etc.) | | |— genome_format.py # Prompt DNA
formatter | | |— genome_library.json # Reusable prompt genome
definitions | | |— tests/ | | |— test_pcl.py | | |—
test_shorthand_codes.py | | |— test_genome_format.py | | |—
layer1_alpha/ # Creative Organism | | |—
alpha_engine.py # Main creative pipeline orchestrator | | |—
intent_parser.py # Intent classification module | | |—
generator.py # Asset generation interface (OpenAI/SD) | | |—
|— evaluator.py # Quality scoring with rubric schema | | |—
|— memory_writer.py # Write approved assets to Zeta memory | | |—
|— improver.py # Iterative improvement loop (max 3x) | | |—
|— tests/ | | |— layer2_beta/ # Governance
Intelligence | | |— beta_council.py # Agent council
orchestrator | | |— quality_scorer.py # Multi-dimensional
scoring engine | | |— identity_guard.py # Identity protection
and brand checker | | |— agents/ | | |— __init__.py | | |—
|— vision_agent.py # Strategic alignment evaluator | | |—
systems_agent.py # Technical integrity evaluator | | |—
culture_agent.py # Identity and culture protection | | |—
innovation_agent.py # Novelty and creativity evaluator | | |— tests/
| | |— layer3_gamma/ # Adaptive Intelligence | | |—
|— gamma_core.py # Learning orchestrator | | |—
knowledge_ingestor.py # External knowledge pipeline | | |—
pattern_engine.py # Pattern discovery and abstraction | | |—
network_learner.py # Federated learning module | | |— tests/
| | |— layer4_delta/ # Research + Simulation | | |—
delta_engine.py # Simulation orchestrator | | |—
scenario_tester.py # Branching outcome evaluator | | |—
digital_twin.py # System replica builder | | |—
future_modeler.py # Probabilistic trajectory modeling | | |—
tests/ | | |— layer5_epsilon/ # Action Layer | | |—
|— epsilon_executor.py # Action orchestrator | | |—
workflow_builder.py # Workflow construction engine | | |—
project_manager.py # Project coordination module | | |—
impl_planner.py # Implementation planning interface | | |—
```

```

tests/ | | | | layer6_zeta/ # Stewardship / Memory |
| | | zeta_store.py # Main memory store interface | | |
lineage_tracker.py # Provenance and lineage tracking | | |
knowledge_graph.py # Knowledge graph manager (Neo4j) | | |
artifact_registry.py # Asset registry and indexing | | | tests/ |
| | | layer7_omega/ # Coordination Layer | | |
omega_coordinator.py # Multi-system orchestrator | | |
priority_matrix.py # Priority arbitration engine | | |
alignment_check.py # Value alignment verification | | | tests/
| | | layer8_genesis/ # Genesis Protocol | | |
genesis.py # Bootstrap sequence controller | | |
identity_init.py # Identity establishment module | | |
mission_config.py # Mission definition and constraints | | |
tests/ | | | venture/ # Venture Modules (9-14) |
| | | module9_cbf/ # Civilization Builder Framework |
| | | cbf_core.py | | | module10_pof/ # Planetary Operating
Framework | | | pof_core.py | | | module11_mil/ #
Market Intelligence Layer | | | mil_core.py | | | module12_com/
# Commercial Operating Manual | | | com_core.py | | | module13_fos/
# Founder Operating System | | | fos_core.py | | | module14_vsf/
# Venture Studio Framework | | | vsf_core.py | | | api/
# REST + GraphQL API surface | | | routes/ | | | prompt.routes.ts
# Layer 0 endpoints | | | asset.routes.ts # Layer 1
endpoints | | | governance.routes.ts # Layer 2 endpoints | | |
| | | venture.routes.ts # Modules 9-14 endpoints | | | middleware/ |
| | | auth.middleware.ts # JWT Bearer validation | | |
rateLimit.middleware.ts # Token-bucket rate limiting | | |
governance.middleware.ts # Auto-route Alpha → Beta | | | server.ts
# Express server entry point | | | db/ #
Database layer | | | schema.sql # Full PostgreSQL DDL
schema | | | migrations/ # Versioned migration scripts |
| | | 001_initial_schema.sql | | | 002_add_knowledge_nodes.sql |
| | | models/ # ORM model definitions |
genome.model.ts | | | asset.model.ts | | | governance.model.ts |
| | | ui/ # Frontend (Next.js 14 / React) |
| | | app/ | | | page.tsx # Root dashboard | | |
layout.tsx | | | api/ # Next.js API routes (thin
wrappers) | | | components/ | | | PromptEngine/ # PCL
input, genome viewer | | | LayerStack/ # Layer status
dashboard | | | GovernanceDashboard/ # Beta council review UI |
| | | VentureModules/ # Module 9-14 UIs | | | styles/ |
| | | globals.css | | | infra/ # Infrastructure as
Code | | | docker/ | | | Dockerfile.api | | | Dockerfile.worker
| | | Dockerfile.ui | | | k8s/ | | | namespaces.yaml | | |
deployments/ | | | ingress.yaml | | | terraform/ | | | main.tf
| | | variables.tf | | | outputs.tf

```

2. Layer-by-Layer Module Breakdown

Each layer of AAES-OS encapsulates a distinct functional responsibility. Layers communicate through well-defined data contracts (schemas) and are independently testable. The inter-layer data flow is: **Layer 0 → 1 → 2 → 6** for the primary creative

pipeline, with **Layers 3, 4, 5** operating in parallel advisory and execution roles, and **Layers 7 and 8** providing coordination and initialization.

2.0 Layer 0 — Prompt Compiler Layer (PCL)

File: `core/layer0_prompt_engine/pcl.py`

The Prompt Compiler Layer (PCL) is the entry point for all human intent into the AAES-OS system. Raw shorthand inputs are parsed, expanded, scored, and serialized into structured **Prompt Genomes** that downstream layers can consume deterministically. The PCL eliminates ambiguity at the point of input and ensures every downstream operation is traceable to a computable specification.

Implementation — `pcl.py`

```
from dataclasses import dataclass, field from typing import List, Optional
from uuid import uuid4 from datetime import datetime, timezone from
shorthand_codes import ShorthandRegistry, ShorthandCode from genome_format
import PromptGenome, GenomeLibrary class PromptCompilerLayer: """
Layer 0: Converts raw shorthand prompt strings into structured
PromptGenome objects for consumption by downstream AAES layers. """
def __init__(self): self.shorthand_registry = ShorthandRegistry()
self.genome_library = GenomeLibrary() def compile(self, raw_input: str)
-> PromptGenome: """ Master compile method. Parses, expands,
scores, stores. Returns a fully-formed PromptGenome. """
tokens: List[ShorthandCode] = self.parse_shorthand_codes(raw_input)
base_text: str = self.extract_base_text(raw_input) genome:
PromptGenome = self.build_genome(tokens, base_text)
genome.fitness_score = self.score_fitness(genome)
self.genome_library.store(genome) return genome def
parse_shorthand_codes(self, text: str) -> List[ShorthandCode]: """
Extract bracketed codes such as [FPS], [HUD], [ARCH] from text.
Returns ordered list of resolved ShorthandCode objects. """
import re pattern = r'\[([A-Z0-9]+)\]' matches =
re.findall(pattern, text) return
[ self.shorthand_registry.resolve(code) for code in
matches if self.shorthand_registry.exists(code) ]
def extract_base_text(self, text: str) -> str: """Strip all shorthand
codes; return the quoted base label.""" import re stripped =
re.sub(r'\[([A-Z0-9]+)\]', '', text).strip() return
stripped.strip('"\'') def build_genome( self, codes:
List[ShorthandCode], base_text: str ) -> PromptGenome:
"""Assemble genome from code expansions and base text."""
expansion_parts = [c.expansion for c in codes] style_modifiers =
list({m for c in codes for m in c.style_modifiers}) traits = [c.code
for c in codes] return
PromptGenome( genome_id=str(uuid4()), name=base_text,
raw_input=base_text, codes=[c.code for c in codes],
base_text=base_text, expansion=" | ".join(expansion_parts),
style=style_modifiers[0] if style_modifiers else "Default",
traits=traits, created_at=datetime.now(timezone.utc).isoformat(),
version=1 ) def score_fitness(self, genome: PromptGenome) ->
float: """ Heuristic fitness scoring: - Code
diversity (+2.0 per unique category represented) - Expansion
richness (+1.0 per 50 chars of expansion) - Base text presence
```

```
(+1.0)          Max score: 10.0          ""          score = 0.0
categories = {          self.shorthand_registry.resolve(c).category
for c in genome.codes          if self.shorthand_registry.exists(c)
}          score += min(len(categories) * 2.0, 5.0)          score +=
min(len(genome.expansion) / 50, 3.0)          if genome.base_text:
score += 1.0          return min(round(score, 1), 10.0)
```

Shorthand Code Schema (JSON)

```
{  "code": "HUD",  "category": "interface",  "expansion": "Cybernetic
interface, telemetry data overlays, sci-fi systems display",
"style_modifiers": ["futuristic", "technical", "overlay"],
"compatible_codes": ["ARCH", "FPS", "SYS"] }
```

Prompt Genome Schema (JSON)

```
{  "genome_id": "550e8400-e29b-41d4-a716-446655440000",  "name": "Wakonda
Grid",  "raw_input": "[HUD][ARCH] 'WAKONDA GRID'",  "codes": ["HUD",
"ARCH"],  "base_text": "WAKONDA GRID",  "expansion": "Cybernetic interface,
telemetry data overlays | Monumental civic architecture, structural
grandeur",  "style": "Regenerative Cyber Architecture",  "traits": ["HUD",
"Biomorphic", "Systems", "Future"],  "fitness_score": 9.2,  "created_at":
"2026-06-16T12:01:00Z",  "version": 1 }
```

Registered Shorthand Codes:

Code	Category	Expansion Summary	Compatible Codes
[FPS]	game	First-person shooter perspective, tactical camera, motion blur	HUD, SYS, ENV
[HUD]	interface	Cybernetic interface, telemetry overlays, sci-fi systems display	ARCH, FPS, SYS
[ARCH]	design	Monumental civic architecture, structural grandeur, spatial drama	HUD, ENV, URB
[SYS]	technical	Systems diagram, network topology, data flow visualization	HUD, ARCH, FPS
[ENV]	environment	Immersive environment design, atmospheric rendering, world-building	ARCH, URB, FPS

Code	Category	Expansion Summary	Compatible Codes
[URB]	urban	Urban landscape, city planning perspective, infrastructure scale	ARCH, ENV, SYS

2.1 Layer 1 — AAES Alpha: Creative Organism

File: core/layer1_alpha/alpha_engine.py

AAES Alpha is the primary creative engine. It receives a `PromptGenome` from Layer 0 and processes it through a six-stage sequential pipeline: parse intent, compile context, generate asset, evaluate quality, write to memory, and iterate for improvement. Each stage is implemented as a discrete class with a single `execute(context)` method, enabling independent testing and hot-swap replacement.

Implementation – alpha_engine.py

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Optional, List, Type
from genome_format import PromptGenome
from layer6_zeta.zeta_store import ZetaStore

@dataclass
class Asset:
    asset_id: str
    asset_type: str # "image" | "document" | "prompt" | "workflow"
    source_genome_id: str
    layer_origin: str
    quality_score: float
    beta_approved: bool
    created_at: str
    stored_in_zeta: bool
    lineage: List[str]
    content_uri: Optional[str] = None

@dataclass
class PipelineContext:
    genome: PromptGenome
    asset: Optional[Asset] = None
    failed: bool = False
    failure_reason: Optional[str] = None
    iteration: int = 0
    max_iterations: int = 3

class PipelineStage(ABC):
    @abstractmethod
    def execute(self, context: PipelineContext) -> PipelineContext:
        pass

class AlphaEngine:
    """
    Layer 1 – AAES Alpha Creative Pipeline.
    Executes six stages sequentially; retries via Improver up to 3x.
    """
    stages: List[Type[PipelineStage]] = [
        IntentParser, ContextCompiler, Generator, Evaluator, MemoryWriter, Improver,
    ]

    def __init__(self):
        self.zeta = ZetaStore()

    def run_pipeline(self, genome: PromptGenome) -> Asset:
        context = PipelineContext(genome=genome)
        for StageClass in self.stages:
            context = StageClass().execute(context)
            if context.failed:
                return self.handle_failure(context)
        return context.asset

    def handle_failure(self, context: PipelineContext) -> Optional[Asset]:
        """Log failure to Zeta audit log; return None or last valid asset."""
        self.zeta.log_failure(context.failure_reason, context.genome)
        return None
```

Asset Schema (JSON)

```
{
  "asset_id": "uuid-v4",
  "type": "image | document | prompt | workflow",
  "source_genome_id": "uuid-v4",
  "layer_origin": "alpha",
  "quality_score": 8.9,
  "beta_approved": true,
  "created_at": "2026-06-16T12:01:00Z",
  "stored_in_zeta": true,
  "lineage": ["parent_asset_id_1", "parent_asset_id_2"],
  "content_uri": "s3://aaes-zeta/assets/uuid-v4"
}
```

Evaluator Rubric Schema (JSON)

```
{
  "rubric_id": "uuid-v4",
  "asset_id": "uuid-v4",
  "criteria": [
    { "name": "Coherence", "weight": 0.25, "score": 8.5 },
    { "name": "Creativity", "weight": 0.20, "score": 9.0 },
    { "name": "Technical Quality", "weight": 0.20, "score": 8.0 },
    { "name": "Mission Alignment", "weight": 0.20, "score": 9.5 },
    { "name": "Governance Compliance", "weight": 0.15, "score": 10.0 }
  ],
  "total_score": 8.9,
  "pass_threshold": 7.5,
  "verdict": "pass",
  "evaluated_at": "2026-06-16T12:01:00Z"
}
```

2.2 Layer 2 – AAES Beta: Governance Intelligence

File: core/layer2_beta/beta_council.py

AAES Beta provides multi-agent governance over all assets produced by Alpha. A council of four specialized agents evaluates each asset from distinct dimensional perspectives. The council then arbitrates verdicts into a final `GovernanceDecision`. All assets produced by Layer 1 must pass through Beta before being stored in Zeta or delivered externally.

Implementation – beta_council.py

```

from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import List, Literal
from datetime import datetime, timezone

@dataclass
class AgentVerdict:
    agent_type: str
    score: float
    verdict: str  # "approved" | "rejected" | "flagged"

notes: str
@dataclass
class GovernanceDecision:
    decision_id: str
    asset_id: str
    verdict: Literal["approved", "rejected", "review_required"]
    agent_verdicts: List[AgentVerdict]
    quality_score: float
    identity_score: float
    timestamp: str
    notes: str
    class

GovernanceAgent(ABC):
    """Abstract base for all Beta council agents."""

@abstractmethod
def evaluate(self, asset) -> AgentVerdict:
    pass

@property
@abstractmethod
def agent_type(self) -> str:
    pass

class VisionAgent(GovernanceAgent):
    """Evaluates strategic and mission alignment."""
    @property
    def agent_type(self) -> str:
        return "vision"
    def evaluate(self, asset) -> AgentVerdict:
        # Checks: Does asset advance AES mission?
        # Checks: Is quality score ≥ 7.5?
        ...

class SystemsAgent(GovernanceAgent):
    """Evaluates technical integrity and schema compliance."""
    @property
    def agent_type(self) -> str:
        return "systems"
    def evaluate(self, asset) -> AgentVerdict:
        # Checks: Schema validity, storage URI reachability, type correctness
        ...

class CultureAgent(GovernanceAgent):
    """Evaluates brand identity and cultural alignment."""
    @property
    def agent_type(self) -> str:
        return "culture"
    def evaluate(self, asset) -> AgentVerdict:
        # Checks: Identity consistency, tone alignment, brand guidelines
        ...

class InnovationAgent(GovernanceAgent):
    """Evaluates novelty, creativity, and differentiation."""
    @property
    def agent_type(self) -> str:
        return "innovation"
    def evaluate(self, asset) -> AgentVerdict:
        # Checks: Uniqueness score vs. genome library, creative leap value
        ...

class BetaCouncil:
    """Layer 2 – AES Beta Governance Council.
    Aggregates four agent verdicts into a binding GovernanceDecision.
    """
    def __init__(self):
        self.agents: List[GovernanceAgent] = [
            VisionAgent(),
            SystemsAgent(),
            CultureAgent(),
            InnovationAgent(),
        ]
    def review(self, asset) -> GovernanceDecision:
        verdicts = [agent.evaluate(asset) for agent in self.agents]
        return self.arbitrate(asset, verdicts)
    def arbitrate(self, asset, verdicts: List[AgentVerdict]) -> GovernanceDecision:

```

```

"""
    Arbitration rules:
    - Any "rejected" with score < 5.0 → "rejected"
    - Otherwise → "review_required"
    """
    avg_score = sum(v.score for v in
verdicts) / len(verdicts)
    if all(v.verdict == "approved" for v in
verdicts):
        final = "approved"
    elif any(v.score < 5.0 for
v in verdicts):
        final = "rejected"
    else:
        final = "review_required"
    identity_v = next(v for v in verdicts if
v.agent_type == "culture")
    from uuid import uuid4
    return
GovernanceDecision(
    decision_id=str(uuid4()),
    asset_id=asset.asset_id,
    verdict=final,
    agent_verdicts=verdicts,
    quality_score=round(avg_score, 2),
    identity_score=identity_v.score,
    timestamp=datetime.now(timezone.utc).isoformat(),
    notes="";
    ".join(v.notes for v in verdicts if v.notes)
)

```

GovernanceDecision Schema (JSON)

```

{
  "decision_id": "uuid-v4",
  "asset_id": "uuid-v4",
  "verdict": "approved
| rejected | review_required",
  "agent_verdicts": [
    { "agent_type":
"vision",
    "score": 9.0, "verdict": "approved", "notes": "" },
    { "agent_type": "systems",
    "score": 8.5, "verdict": "approved", "notes":
"" },
    { "agent_type": "culture",
    "score": 9.5, "verdict": "approved",
"notes": "" },
    { "agent_type": "innovation", "score": 8.7, "verdict":
"approved", "notes": "" }
  ],
  "quality_score": 8.93,
  "identity_score":
9.5,
  "timestamp": "2026-06-16T12:01:00Z",
  "notes": ""
}

```

2.3 Layer 3 — AAES Gamma: Adaptive Intelligence

File: core/layer3_gamma/gamma_core.py

AAES Gamma implements the system's capacity for continuous learning. It observes system events, abstracts recurring patterns, constructs knowledge models, applies those models to improve system behavior, validates outcomes, and stores validated knowledge to Zeta. The learning loop is event-driven and operates asynchronously alongside the primary creative pipeline.

Implementation – gamma_core.py

```

from dataclasses import dataclass, field
from typing import List, Optional, Any
@dataclass class SystemEvent:
    event_id: str
    event_type: str
    # "asset_created" | "governance_decision" | "user_feedback"
    layer_source: str
    payload: dict
    timestamp: str
@dataclass class Pattern:
    pattern_id: str
    description: str
    supporting_events: List[str]
    # event_ids
    confidence: float
    # 0.0 - 1.0
    frequency: int
@dataclass class KnowledgeModel:
    model_id: str
    derived_from_pattern: str
    # pattern_id
    rules: List[str]
    applicability_scope: List[str]
    # layer names
    confidence: float
@dataclass class ValidationResult:
    valid: bool
    improvement_delta: float
    # positive = improvement
    notes: str
class GammaCore:
    """
    Layer 3
    – AAES Gamma Adaptive Learning Loop.
    Continuously refines system behavior
    through pattern abstraction.
    """
    def observe(self, event:
SystemEvent) -> None:
        """Ingest and index a new system event for
pattern analysis."""
        self._event_buffer.append(event)
        if
len(self._event_buffer) >= self._batch_size:
            pattern =
self.abstract(self._event_buffer)
            model = self.model(pattern)
            self.apply(model)
        def abstract(self, observations: List[SystemEvent]) ->
Pattern:
            """Identify recurring structures across a batch of
events."""
            ...
        def model(self, pattern: Pattern) ->

```



```
KnowledgeModel: """Convert an abstract pattern into an actionable
knowledge model."""
... def apply(self, model: KnowledgeModel) -
> Any: """Inject model guidance into the relevant system layers."""
... def validate(self, application: Any) -> ValidationResult:
"""Measure the impact of an applied knowledge model."""
... def
store(self, result: ValidationResult) -> None: """Persist validated
knowledge nodes to Zeta store."""
...
```

KnowledgeNode Schema (JSON)

```
{ "node_id": "uuid-v4", "type": "concept | pattern | rule | insight",
"content": "High fitness_score genomes with 3+ codes consistently yield
quality_score > 8.5 in Alpha", "source_layer": "gamma", "confidence":
0.87, "connections": ["node_id_a", "node_id_b"], "last_updated": "2026-
06-16T12:01:00Z" }
```

2.4 Layer 4 — AAES Delta: Research + Simulation

File: core/layer4_delta/delta_engine.py

AAES Delta provides the system with foresight. It builds scenarios from hypotheses, runs probabilistic simulations across branching outcome trees, constructs digital twins of subsystems for safe experimentation, and produces trajectory models for future state forecasting. Delta outputs inform Epsilon (Layer 5) action planning.

Implementation – delta_engine.py

```
from dataclasses import dataclass, field from typing import List, Dict, Any,
Optional @dataclass class Scenario: scenario_id: str hypothesis:
str variables: List[Dict[str, Any]] branches: List[Dict[str, str]]
# {"condition": "...", "outcome": "..."} confidence_intervals: Dict[str,
float] simulation_runs: int result_summary: Optional[str] = None
@dataclass class SimulationResult: result_id: str scenario_id: str
outcomes: List[Dict[str, Any]] winning_branch: str confidence: float
recommendations: List[str] @dataclass class DigitalTwin: twin_id: str
source_system: str state_snapshot: Dict[str, Any] replica_accuracy:
float # 0.0 – 1.0 @dataclass class Trajectory:
trajectory_id: str start_state: Dict[str, Any] steps: int
projected_states: List[Dict[str, Any]] probability_mass: List[float]
class DeltaEngine: """ Layer 4 – AAES Delta Research + Simulation
Engine. """
def create_scenario(self, hypothesis: str) -> Scenario:
"""Formalize a hypothesis into a testable scenario structure."""
...
def run_simulation( self, scenario: Scenario, runs:
int = 1000 ) -> SimulationResult: """Execute Monte Carlo
simulation across scenario branches."""
... def
build_digital_twin( self, system_spec: Dict[str, Any] ) -
> DigitalTwin: """Construct a stateful replica of a target
subsystem."""
... def model_future( self,
current_state: Dict[str, Any], steps: int ) -> Trajectory:
"""Project probabilistic future states from current system
state."""
...
```

Scenario Schema (JSON)

```
{ "scenario_id": "uuid-v4", "hypothesis": "If genome fitness_score
threshold is raised to 8.0, does Alpha quality increase?", "variables": [
{ "name": "fitness_threshold", "type": "float", "range": [6.0, 10.0] }, {
"name": "iteration_limit", "type": "int", "range": [1, 5] } ],
"branches": [ { "condition": "fitness_threshold >= 8.0", "outcome":
"Higher average quality_score, fewer assets produced" }, { "condition":
```

```
"fitness_threshold < 8.0", "outcome": "Higher volume, lower average quality"
} ], "confidence_intervals": { "quality_improvement": 0.85,
"volume_reduction": 0.72 }, "simulation_runs": 1000, "result_summary":
"Raising threshold to 8.0 improves mean quality_score by 0.8 points at 85%
confidence." }
```

2.5 Layer 5 — AAES Epsilon: Action Layer

File: `core/layer5_epsilon/epsilon_executor.py`

AAES Epsilon converts simulation-validated plans into real-world executable workflows and projects. It is the only layer authorized to trigger external actions (API calls, file writes, notifications, deployments). All Epsilon actions are logged to Zeta's audit trail and must be pre-cleared by Omega (Layer 7) for alignment.

Implementation – `epsilon_executor.py`

```
from dataclasses import dataclass, field from typing import List, Optional,
Literal from uuid import uuid4 from datetime import datetime, timezone
@dataclass class WorkflowStep:    step_id: int    action: str    inputs:
List[str]    outputs: List[str]    status: Literal["pending", "running",
"complete", "failed"]    started_at: Optional[str] = None    completed_at:
Optional[str] = None    @dataclass class Workflow:    workflow_id: str
name: str    source_scenario_id: str    steps: List[WorkflowStep]
status: Literal["draft", "active", "complete", "failed"]    created_at: str
@dataclass class Project:    project_id: str    workflow_id: str    owner:
str    milestones: List[str]    status: str    @dataclass class
ImplementationPlan:    plan_id: str    project_id: str    phases:
List[dict]    resource_requirements: dict    estimated_duration_days: int
@dataclass class ActionResult:    action_id: str    step_id: int
success: bool    output: dict    error: Optional[str] = None    class
EpsilonExecutor:    """    Layer 5 – AAES Epsilon Action Orchestrator.
All external side effects originate here.    """    def
build_workflow(self, plan: "SimulationResult") -> Workflow:
"""Convert a simulation result into an executable workflow."""    ...
def create_project(self, workflow: Workflow) -> Project:    """Wrap a
workflow in project management scaffolding."""    ...    def
generate_impl_plan(self, project: Project) -> ImplementationPlan:
"""Produce a phased implementation plan for a project."""    ...
def execute_action(self, action: str, step: WorkflowStep) -> ActionResult:
"""    Execute a single workflow step action.    Logs to Zeta audit
trail before and after execution.    """    ...
```

Workflow Schema (JSON)

```
{  "workflow_id": "uuid-v4",  "name": "Deploy Genome Library Starter Kit",
"source_scenario_id": "uuid-v4",  "steps": [    {    "step_id": 1,
"action": "package_genome_library",    "inputs": ["genome_library.json"],
"outputs": ["starter_kit_v1.zip"],    "status": "complete"    },    {
"step_id": 2,    "action": "upload_to_s3",    "inputs":
["starter_kit_v1.zip"],    "outputs": ["s3://aaes-products/starter-kit-
v1.zip"],    "status": "pending"    },    {    "step_id": 3,
"action": "publish_gumroad_product",    "inputs":
["s3://aaes-products/starter-kit-v1.zip"],    "outputs":
["gumroad_product_id"],    "status": "pending"    }  ],  "status":
"active",  "created_at": "2026-06-16T12:01:00Z" }
```

2.6 Layer 6 — AAES Zeta: Stewardship

File: core/layer6_zeta/zeta_store.py

AAES Zeta is the system's long-term memory, provenance engine, and knowledge graph backbone. Every asset, decision, knowledge node, scenario, workflow, and audit entry is persisted through Zeta. It provides immutable lineage chains, semantic retrieval via vector embeddings, and full knowledge graph traversal via Neo4j integration.

Implementation – zeta_store.py

```
from typing import List, Optional
from dataclasses import dataclass

@dataclass class ProvenanceChain:
    asset_id: str
    ancestors: List[str]
    # ordered from oldest to newest
    creation_events: List[dict] # timestamps
+ layer sources
    depth: int
@dataclass class KnowledgeGraph:
    nodes: List[dict]
    edges: List[dict]
    node_count: int
    edge_count: int

class ZetaStore:
    """
    Layer 6 – AAES Zeta Stewardship and Memory Store.
    Interfaces: PostgreSQL (structured), Redis (hot cache),
    Neo4j (graph), S3 (binary assets), Pinecone (vectors).
    """
    def store_asset(self, asset) -> str:
        """Persist asset metadata to PostgreSQL. Upload binary content to S3. Index embedding to Pinecone. Returns: asset_id"""
    def retrieve_asset(self, asset_id: str):
        """Fetch asset metadata from PostgreSQL; presign S3 URI."""
    def track_lineage(self, asset, parent_id: str) -> None:
        """Append a lineage edge to the artifact's provenance chain."""
    def build_knowledge_graph(self) -> KnowledgeGraph:
        """Query Neo4j and return full knowledge graph snapshot."""
    def get_provenance(self, asset_id: str) -> ProvenanceChain:
        """Recursively resolve full ancestor chain for an asset."""
    def log_failure(self, reason: str, genome) -> None:
        """Write pipeline failure event to the audit_log table."""
    def semantic_search(self, query: str, top_k: int = 10) -> List[dict]:
        """Embed query via OpenAI embeddings API. Query Pinecone for top_k nearest asset embeddings. Returns ranked list of asset metadata. """
```

ArtifactRecord Schema (JSON)

```
{
  "artifact_id": "uuid-v4",
  "title": "Wakonda Grid – HUD Render v1",
  "type": "image | document | prompt | workflow | spec",
  "layer_origin": "alpha",
  "quality_score": 8.9,
  "beta_approved": true,
  "created_at": "2026-06-16T12:01:00Z",
  "lineage_chain": ["parent_id_1", "parent_id_2"],
  "tags": ["HUD", "ARCH", "Wakonda", "v1"],
  "storage_uri": "s3://aaes-zeta/artifacts/uuid-v4",
  "status": "active | archived | deprecated"
}
```

2.7 Layer 7 — AAES Omega: Coordination

File: core/layer7_omega/omega_coordinator.py

AAES Omega acts as the system-wide arbiter and alignment verifier. It monitors all active layers, resolves objective conflicts between layers, verifies that every consequential decision is aligned with core system values, and maintains an

immutable audit record of all resolutions. Omega is the last gate before any Layer 5 action is authorized.

Implementation – omega_coordinator.py

```
from dataclasses import dataclass, field from typing import List, Literal,
Optional from uuid import uuid4 from datetime import datetime, timezone
@dataclass class ObjectiveConflict:    conflict_id: str
competing_objectives: List[str]    originating_layers: List[str]
severity: Literal["low", "medium", "high", "critical"]    @dataclass class
Resolution:    conflict_id: str    resolution_strategy: Literal["priority",
"weighted", "defer"]    selected_objective: str    rationale: str
alignment_score: float    @dataclass class AlignmentScore:    decision_id:
str    score: float    # 0.0 – 10.0; threshold = 7.0    passed: bool
flags: List[str]    @dataclass class AuditEntry:    entry_id: str    action:
str    layer: str    actor: str    resource_id: str    payload: dict
timestamp: str    @dataclass class OrchestratorState:    active_layers:
List[str]    pending_conflicts: List[ObjectiveConflict]    alignment_score:
float    last_audit: str    class OmegaCoordinator:    """    Layer 7 –
AAES Omega Coordination Engine.    Alignment gate and conflict resolution
authority.    """    def orchestrate(self, systems: list) ->
OrchestratorState:    """Poll all active layer states; identify
conflicts."""    ...    def arbitrate(    self,
conflicts: List[ObjectiveConflict]    ) -> List[Resolution]:
"""Apply resolution strategy to each conflict in priority
order."""    ...    def check_alignment(self, decision) ->
AlignmentScore:    """    Score a proposed decision against AAES
core values:    - Mission advancement    - Identity integrity
- Governance compliance    - Innovation contribution
Threshold: 7.0. Below threshold -> blocked.    """    ...    def
audit(self, action: str, layer: str, resource_id: str, payload: dict) ->
AuditEntry:    """Write immutable audit entry to Zeta audit_log."""
entry = AuditEntry(    entry_id=str(uuid4()),
action=action,    layer=layer,    actor="system",
resource_id=resource_id,    payload=payload,
timestamp=datetime.now(timezone.utc).isoformat()    )
self._zeta.persist_audit(entry)    return entry
```

ConflictResolution Schema (JSON)

```
{  "conflict_id": "uuid-v4",  "competing_objectives": [    "gamma: ingest
external dataset immediately",    "epsilon: use current dataset for active
workflow"  ],  "resolution_strategy": "priority",  "selected_objective":
"epsilon: use current dataset for active workflow",  "rationale": "Active
workflow execution takes precedence over async learning ingestion per
priority matrix tier 1.",  "alignment_score": 9.1,  "logged_at": "2026-06-
16T12:01:00Z" }
```

2.8 Layer 8 — Genesis Protocol

File: core/layer8_genesis/genesis.py

The Genesis Protocol is the system bootstrap sequence. It is executed once at initial deployment and again after any catastrophic state reset. It establishes system identity, loads the mission configuration, activates governance, initializes Zeta memory, deploys all agents, and enables the evolutionary learning loop. Genesis is idempotent — repeated execution is safe and produces consistent initial state.

Implementation – genesis.py

```
from dataclasses import dataclass, field from typing import Optional, List
from datetime import datetime, timezone @dataclass class GenesisConfig:
system_name: str version: str founder: str mission: str
governance_rules: dict memory_backend: str #
"postgresql+redis" agent_config: dict evolution_enabled: bool
@dataclass class SystemState: identity_established: bool = False
mission_loaded: bool = False governance_active: bool = False
memory_initialized: bool = False agents_deployed: bool = False
evolution_enabled: bool = False bootstrap_completed_at: Optional[str] =
None errors: List[str] = field(default_factory=list) class
GenesisProtocol: """ Layer 8 – AAES Genesis Bootstrap Protocol.
Activation sequence is strictly ordered; each step is a prerequisite for
all subsequent steps. Failures halt and log to stderr. """
ACTIVATION_SEQUENCE = [ "establish_identity", "set_mission",
"activate_governance", "init_memory", "deploy_agents",
"enable_evolution", ] def bootstrap(self, config: GenesisConfig) ->
SystemState: state = SystemState() for step_name in
self.ACTIVATION_SEQUENCE: try: step_fn =
getattr(self, step_name) state = step_fn(state, config)
except Exception as exc: state.errors.append(f"{step_name}:
{str(exc)}") break if not state.errors:
state.bootstrap_completed_at = datetime.now(timezone.utc).isoformat()
return state def establish_identity(self, state: SystemState, config:
GenesisConfig) -> SystemState: """Load system name, version, and
founder identity from config.""" state.identity_established = True
return state def set_mission(self, state: SystemState, config:
GenesisConfig) -> SystemState: """Persist mission statement and
constraint ruleset to Zeta.""" state.mission_loaded = True
return state def activate_governance(self, state: SystemState, config:
GenesisConfig) -> SystemState: """Initialize Beta council agents and
identity guard.""" state.governance_active = True return
state def init_memory(self, state: SystemState, config: GenesisConfig) -
> SystemState: """Establish PostgreSQL schema, Redis cache, Neo4j
graph, Pinecone index.""" state.memory_initialized = True
return state def deploy_agents(self, state: SystemState, config:
GenesisConfig) -> SystemState: """Instantiate all layer engines and
register with Omega coordinator.""" state.agents_deployed = True
return state def enable_evolution(self, state: SystemState, config:
GenesisConfig) -> SystemState: """Activate Gamma learning loop if
config.evolution_enabled is True.""" state.evolution_enabled =
config.evolution_enabled return state
```

GenesisConfig Schema (JSON)

```
{ "system_name": "AAES-OS", "version": "1.0.0", "founder": "Jon
Halstead", "mission": "Convert human intent into generated assets,
evaluated knowledge, evolving systems, and commercial products.",
"governance_rules": { "min_quality_score": 7.5,
"identity_guard_enabled": true, "require_beta_approval": true,
"audit_all_actions": true }, "memory_backend": "postgresql+redis",
"agent_config": { "vision_agent": { "enabled": true },
"systems_agent": { "enabled": true }, "culture_agent": { "enabled":
true }, "innovation_agent": { "enabled": true } },
"evolution_enabled": true }
```

3. API Surface Definition

3.1 REST API Endpoints

All endpoints are prefixed with `/api/v1/`. Requests require a valid JWT Bearer token in the `Authorization` header. All write operations are automatically audited via the Omega coordinator.

Method	Endpoint	Description	Layer	Auth
POST	<code>/api/v1/prompt/compile</code>	Compile shorthand prompt string to Prompt Genome	Layer 0	Required
POST	<code>/api/v1/prompt/genome</code>	Create and store a new genome manually	Layer 0	Required
GET	<code>/api/v1/prompt/genome/:id</code>	Retrieve genome by UUID	Layer 0	Required
GET	<code>/api/v1/prompt/genomes</code>	List all genomes (paginated)	Layer 0	Required
POST	<code>/api/v1/alpha/generate</code>	Run full Alpha creative pipeline from genome	Layer 1	Required
GET	<code>/api/v1/alpha/asset/:id</code>	Retrieve generated asset metadata and content URI	Layer 1	Required
POST	<code>/api/v1/beta/review</code>	Submit asset for full governance council review	Layer 2	Required
GET	<code>/api/v1/beta/decision/:id</code>	Retrieve governance decision by UUID	Layer 2	Required
GET	<code>/api/v1/</code>	List all	Layer 2	Required

Method	Endpoint	Description	Layer	Auth
	beta/ decisions	governance decisions (filterable by verdict)		
POST	/api/v1/ gamma/ingest	Ingest external knowledge document into Gamma pipeline	Layer 3	Required
GET	/api/v1/ gamma/nodes	List knowledge nodes (filterable by type, confidence)	Layer 3	Required
POST	/api/v1/ delta/ simulate	Create scenario and run simulation	Layer 4	Required
GET	/api/v1/ delta/ scenario/:id	Retrieve simulation scenario and results	Layer 4	Required
POST	/api/v1/ delta/twin	Build a digital twin from system specification	Layer 4	Required
POST	/api/v1/ epsilon/ workflow	Create a new executable workflow from simulation result	Layer 5	Required
POST	/api/v1/ epsilon/ execute	Execute a specific workflow step action	Layer 5	Required
GET	/api/v1/ epsilon/ workflow/:id	Retrieve workflow state and step statuses	Layer 5	Required
GET	/api/v1/ zeta/ artifact/:id	Retrieve artifact metadata and presigned S3	Layer 6	Required

Method	Endpoint	Description	Layer	Auth
		URI		
GET	/api/v1/zeta/lineage/:id	Get full provenance chain for an artifact	Layer 6	Required
POST	/api/v1/zeta/search	Semantic search across all stored artifacts	Layer 6	Required
GET	/api/v1/omega/status	Retrieve current coordination and alignment state	Layer 7	Required
GET	/api/v1/omega/audit	Query audit log (filterable by layer, actor, date)	Layer 7	Required
POST	/api/v1/genesis/bootstrap	Initialize or re-initialize the AAES-OS system	Layer 8	Admin Only
GET	/api/v1/genesis/state	Get current bootstrap state and layer health	Layer 8	Required

3.2 Authentication and Middleware

Middleware	File	Behavior
Auth	auth.middleware.ts	Validates JWT Bearer token on all routes. Rejects 401 on missing or invalid token. Admin-scoped endpoints require role: admin claim.
Rate Limit	rateLimit.middleware.ts	Standard tier: 100 req/min per API key. Enterprise tier: 1,000 req/min. Uses Redis token bucket. Returns

Middleware	File	Behavior
		429 with Retry-After header on breach.
Governance	<code>governance.middleware.ts</code>	All routes under <code>/api/v1/alpha/</code> are automatically forwarded to Beta council for review before response is returned. Rejected assets return 422 with decision payload.
Audit Logger	Inline in all write routes	All POST, PUT, DELETE operations emit an audit event to <code>OmegaCoordinator.audit()</code> before the route handler returns. Failures in audit logging do not block the primary operation but are flagged in monitoring.

NOTE — Governance Middleware Behavior

The governance middleware on `/alpha/` routes operates synchronously for POST `/generate` calls (blocking until Beta review completes). For asynchronous asset generation jobs, a webhook callback URL may be provided in the request body; the middleware will POST the `GovernanceDecision` to the callback once review concludes.

4. Database Schema

4.1 PostgreSQL DDL — Full Schema

```
-- ===== -- AES-
OS PostgreSQL Schema v1.0 -- Author: Jon Halstead | Created: 2026-06-16 --
===== -- Enable
UUID generation CREATE EXTENSION IF NOT EXISTS "pgcrypto"; --
----- -- Table:
prompt_genomes --
----- CREATE TABLE
```

```

prompt_genomes (      id          UUID PRIMARY KEY DEFAULT
gen_random_uuid(),   name          VARCHAR(255) NOT NULL,      raw_input
TEXT NOT NULL,       codes         TEXT[] NOT NULL DEFAULT '{}',
expansion            TEXT,         style          VARCHAR(255),      traits
TEXT[] DEFAULT '{}', fitness_score  NUMERIC(4,2) CHECK (fitness_score
BETWEEN 0 AND 10),   created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
version              INTEGER NOT NULL DEFAULT 1,      CONSTRAINT chk_fitness CHECK
(fitness_score >= 0 AND fitness_score <= data-id="453" 10) ); CREATE INDEX
idx_genomes_fitness  ON prompt_genomes (fitness_score DESC); CREATE INDEX
idx_genomes_codes    ON prompt_genomes USING GIN (codes); --
----- -- Table:
assets -- -----
CREATE TABLE assets (      id          UUID PRIMARY KEY DEFAULT
gen_random_uuid(),       type          VARCHAR(50) NOT NULL CHECK (type IN
('image', 'document', 'prompt', 'workflow', 'spec')),      genome_id      UUID
REFERENCES prompt_genomes(id) ON DELETE SET NULL,      layer_origin
VARCHAR(50) NOT NULL DEFAULT 'alpha',      quality_score  NUMERIC(4,2) CHECK
(quality_score BETWEEN 0 AND 10),      beta_approved  BOOLEAN NOT NULL
DEFAULT FALSE,      created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
storage_uri           TEXT,      status          VARCHAR(20) NOT NULL DEFAULT
'active'
CHECK (status IN
('active', 'archived', 'deprecated')) ); CREATE INDEX idx_assets_genome_id
ON assets (genome_id); CREATE INDEX idx_assets_beta_approved  ON assets
(beta_approved); CREATE INDEX idx_assets_status              ON assets (status);
CREATE INDEX idx_assets_quality              ON assets (quality_score DESC); --
----- -- Table:
governance_decisions --
----- CREATE TABLE
governance_decisions (      id          UUID PRIMARY KEY DEFAULT
gen_random_uuid(),      asset_id      UUID NOT NULL REFERENCES assets(id) ON
DELETE CASCADE,      verdict          VARCHAR(20) NOT NULL
CHECK (verdict IN ('approved', 'rejected', 'review_required')),
quality_score  NUMERIC(4,2),      identity_score  NUMERIC(4,2),
agent_verdicts JSONB NOT NULL DEFAULT '[]',      timestamp      TIMESTAMPTZ
NOT NULL DEFAULT NOW(),      notes          TEXT ); CREATE INDEX
idx_gov_asset_id ON governance_decisions (asset_id); CREATE INDEX
idx_gov_verdict  ON governance_decisions (verdict); --
----- -- Table:
knowledge_nodes --
----- CREATE TABLE
knowledge_nodes (      id          UUID PRIMARY KEY DEFAULT
gen_random_uuid(),      type          VARCHAR(50) NOT NULL
CHECK (type IN ('concept', 'pattern', 'rule', 'insight')),      content
TEXT NOT NULL,      source_layer  VARCHAR(50) NOT NULL,      confidence
NUMERIC(5,4) CHECK (confidence BETWEEN 0 AND 1),      connections  JSONB
NOT NULL DEFAULT '[]',      last_updated  TIMESTAMPTZ NOT NULL DEFAULT NOW()
); CREATE INDEX idx_kn_type              ON knowledge_nodes (type); CREATE INDEX
idx_kn_confidence  ON knowledge_nodes (confidence DESC); CREATE INDEX
idx_kn_source      ON knowledge_nodes (source_layer); --
----- -- Table:
scenarios -- -----
CREATE TABLE scenarios (      id          UUID PRIMARY KEY DEFAULT
gen_random_uuid(),      hypothesis    TEXT NOT NULL,      variables
JSONB NOT NULL DEFAULT '[]',      branches      JSONB NOT NULL DEFAULT '[]',
confidence_intervals JSONB,      simulation_runs INTEGER DEFAULT 1000,
result_summary       TEXT,      created_at    TIMESTAMPTZ NOT NULL DEFAULT
NOW() ); -- -----
-- Table: workflows --
----- CREATE TABLE
workflows (      id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

name          VARCHAR(255) NOT NULL,      scenario_id    UUID REFERENCES
scenarios(id) ON DELETE SET NULL,      steps          JSONB NOT NULL DEFAULT
'[]',      status          VARCHAR(20) NOT NULL DEFAULT 'draft'
CHECK (status IN ('draft','active','complete','failed')),      created_at
TIMESTAMPTZ NOT NULL DEFAULT NOW() ); CREATE INDEX idx_workflows_status ON
workflows (status); --
----- -- Table:
artifacts -- -----
CREATE TABLE artifacts (      id          UUID PRIMARY KEY DEFAULT
gen_random_uuid(),      title          VARCHAR(255) NOT NULL,      type
VARCHAR(50) NOT NULL,      layer_origin  VARCHAR(50),      quality_score
NUMERIC(4,2),      lineage_chain  JSONB NOT NULL DEFAULT '[]',      tags
JSONB NOT NULL DEFAULT '[]',      storage_uri    TEXT,      status
VARCHAR(20) NOT NULL DEFAULT 'active' CHECK (status IN
('active','archived','deprecated')),      created_at    TIMESTAMPTZ NOT NULL
DEFAULT NOW() ); CREATE INDEX idx_artifacts_status ON artifacts (status);
CREATE INDEX idx_artifacts_tags    ON artifacts USING GIN (tags); --
----- -- Table:
audit_log -- -----
CREATE TABLE audit_log (      id          UUID PRIMARY KEY DEFAULT
gen_random_uuid(),      action          VARCHAR(255) NOT NULL,      layer
VARCHAR(50) NOT NULL,      actor          VARCHAR(255) NOT NULL DEFAULT
'system',      resource_id    UUID,      payload          JSONB NOT NULL
DEFAULT '{}',      timestamp      TIMESTAMPTZ NOT NULL DEFAULT NOW() );
CREATE INDEX idx_audit_layer      ON audit_log (layer); CREATE INDEX
idx_audit_timestamp ON audit_log (timestamp DESC); CREATE INDEX
idx_audit_actor      ON audit_log (actor); CREATE INDEX idx_audit_resource
ON audit_log (resource_id);

```

4.2 Redis Usage

Key Pattern	Data Type	Purpose	TTL
session:{user_id}	Hash	Active user session data (JWT payload, preferences)	24h
genome:hot:{genome_id}	JSON String	Hot-cached genome objects (top 500 by fitness_score)	1h
ratelimit:{api_key}:{window}	Integer	Request counter for token-bucket rate limiting	60s
omega:state	JSON String	Real-time OrchestratorState for Omega coordinator	30s (auto-refresh)
beta:queue	List	Pending asset IDs awaiting governance	No TTL

Key Pattern	Data Type	Purpose	TTL
		review	
alpha:pipeline:{job_id}	Hash	Real-time pipeline stage progress for async jobs	2h

5. Technology Stack Matrix

Component	Technology	Version	Purpose	Layer(s)
Core Engine	Python	3.12	All layer logic, agents, pipelines, learning loops	0-8
API Server	Node.js + TypeScript + Express	Node 22 / TS 5.4	REST API surface, middleware stack, routing	API
Frontend	Next.js + React + TailwindCSS	Next.js 14 / React 18	Dashboard UI, governance viewer, venture module UIs	UI
Primary Database	PostgreSQL	16	Persistent structured storage for all entities	All
Cache / State	Redis	7.2	Hot cache, sessions, real-time Omega state, rate limiting	0, 2, 7
AI Generation	OpenAI API	GPT-4o / Embeddings v3	Layer 1 text generation, Zeta semantic embeddings	1, 6
Image Generation	Stable Diffusion (via Replicate)	SDXL / SD3	Layer 1 image asset generation	1

Component	Technology	Version	Purpose	Layer(s)
Knowledge Graph	Neo4j	5.x	Gamma pattern mapping, Zeta provenance graph traversal	3, 6
Vector Store	Pinecone / pgvector	Pinecone v3	Semantic search across assets and knowledge nodes	6
File Storage	AWS S3	—	Binary asset storage (images, documents, exports)	1, 6
Orchestration	Docker + Kubernetes	Docker 26 / K8s 1.30	Container orchestration, scaling, service mesh	Infra
CI/CD	GitHub Actions	—	Automated build, test, lint, and deploy pipeline	Infra
Monitoring	Prometheus + Grafana	Prometheus 2.x	System health, layer metrics, SLA dashboards	Infra
Authentication	Auth0 + JWT	—	User authentication , role-based authorization, API keys	API
ORM (Python)	SQLAlchemy	2.x	Database access layer for Python core modules	0–8
ORM (TypeScript)	Prisma	5.x	Database access layer for API server	API
Task Queue	Celery + Redis	Celery 5.x	Async pipeline execution, Gamma	1, 3, 5

Component	Technology	Version	Purpose	Layer(s)
			learning loops	
Logging	Structlog + CloudWatch	—	Structured JSON logging across all layers	All

6. Environment Variables and Configuration

Copy `.env.example` to `.env` at repository root. Never commit `.env` to source control. All secrets must be rotated quarterly. Production values are managed via AWS Secrets Manager and injected into Kubernetes pods at runtime.

```
# ===== # AAES-OS
Environment Configuration # File: .env.example # Author: Jon Halstead |
Version: 1.0.0 | 2026-06-16 #
===== # Copy this
file to .env and populate all values before running. # DO NOT commit .env to
version control. #
===== #
----- # DATABASE
CONFIGURATION #
-----
POSTGRES_URL=postgresql://aaes_user:password@localhost:5432/aaes_os
POSTGRES_TEST_URL=postgresql://aaes_user:password@localhost:5432/aaes_os_test
POSTGRES_POOL_SIZE=20 POSTGRES_MAX_OVERFLOW=10
REDIS_URL=redis://localhost:6379/0 REDIS_SESSION_DB=1 REDIS_CACHE_DB=2
NEO4J_URI=bolt://localhost:7687 NEO4J_USERNAME=neo4j NEO4J_PASSWORD=changeme
# ----- # AI / ML
API KEYS # -----
OPENAI_API_KEY=sk-... OPENAI_EMBEDDING_MODEL=text-embedding-3-large
OPENAI_GENERATION_MODEL=gpt-4o REPLICATE_API_KEY=r8_...
STABLE_DIFFUSION_MODEL=stability-ai/sd-xl:latest PINECONE_API_KEY=pcsk_...
PINECONE_INDEX_NAME=aaes-zeta-embeddings PINECONE_ENVIRONMENT=us-east-1-aws
# ----- # AWS
CONFIGURATION #
-----
AWS_ACCESS_KEY_ID=AKIA... AWS_SECRET_ACCESS_KEY=... AWS_REGION=us-east-1
S3_BUCKET_ASSETS=aaes-zeta-assets S3_BUCKET_PRODUCTS=aaes-products
S3_PREIGNED_URL_EXPIRY=3600 ECR_REGISTRY=123456789012.dkr.ecr.us-east-
1.amazonaws.com #
----- #
AUTHENTICATION #
-----
AUTH0_DOMAIN=aaes-os.us.auth0.com AUTH0_CLIENT_ID=... AUTH0_CLIENT_SECRET=...
AUTH0_AUDIENCE=https://api.aaes-os.com JWT_SECRET=super-secret-jwt-key-min-
32-chars JWT_EXPIRY=86400 #
----- # SYSTEM
CONFIGURATION #
-----
```

```

AAES_VERSION=1.0.0 AAES_ENV=development # development | staging |
production GENESIS_CONFIG_PATH=core/layer8_genesis/genesis_config.json
GOVERNANCE_STRICT_MODE=true # false = log-only, true = blocking
MIN_QUALITY_THRESHOLD=7.5 MAX_PIPELINE_ITERATIONS=3
OMEGA_ALIGNMENT_THRESHOLD=7.0 #
----- # API SERVER
# -----
API_PORT=4000 API_BASE_URL=http://localhost:4000
CORS_ORIGINS=http://localhost:3000,https://app.aaes-os.com
RATE_LIMIT_STANDARD=100 RATE_LIMIT_ENTERPRISE=1000 #
----- # FRONTEND #
-----
NEXT_PUBLIC_API_URL=http://localhost:4000 NEXT_PUBLIC_APP_NAME=AAES-OS
NEXT_PUBLIC_VERSION=1.0.0 #
----- # CELERY /
TASK QUEUE # -----
CELERY_BROKER_URL=redis://localhost:6379/3
CELERY_RESULT_BACKEND=redis://localhost:6379/4 CELERY_WORKER_CONCURRENCY=4 #
----- # MONITORING
# -----
PROMETHEUS_PORT=9090 GRAFANA_PORT=3001 SENTRY_DSN=https://...@sentry.io/...
# ----- # FEATURE
FLAGS # -----
ENABLE_GAMMA_LEARNING=true ENABLE_DELTA_SIMULATION=true
ENABLE_VENTURE_MODULES=false # Modules 9-14 (Phase 5+)
ENABLE_FEDERATED_LEARNING=false # Layer 3 network_learner.py
ENABLE_DIGITAL_TWINS=false # Layer 4 digital_twin.py
ENABLE_SEMANTIC_SEARCH=true # Zeta Pinecone search
ENABLE_ASYNC_GOVERNANCE=false # Async Beta review (webhook mode)

```

7. Deployment Architecture

7.1 Docker Compose Services

```

version: "3.9" services:
  aaes-api:
    build:
      context: .
      dockerfile: infra/docker/Dockerfile.api
    ports:
      - "4000:4000"
    environment:
      - NODE_ENV=development
    env_file: .env
    depends_on:
      - postgres
      - redis
    restart: unless-stopped
  aaes-worker:
    build:
      context: .
      dockerfile: infra/docker/Dockerfile.worker
    command: celery -A core.worker worker --loglevel=info --concurrency=4
    env_file: .env
    depends_on:
      - postgres
      - redis
    restart: unless-stopped
  aaes-ui:
    build:
      context: .
      dockerfile: infra/docker/Dockerfile.ui
    ports:
      - "3000:3000"
    env_file: .env
    depends_on:
      - aaes-api
    restart: unless-stopped
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: aaes_os
      POSTGRES_USER: aaes_user
      POSTGRES_PASSWORD: password
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./db/schema.sql:/docker-entrypoint-initdb.d/01_schema.sql
    ports:
      - "5432:5432"
  redis:
    image: redis:7.2-alpine
    command: redis-server --maxmemory 512mb --maxmemory-policy allkeys-lru
    ports:
      - "6379:6379"
    volumes:
      - redisdata:/data
  neo4j:
    image: neo4j:5-community
    environment:
      NEO4J_AUTH: neo4j/changeme
    ports:
      - "7474:7474"
      - "7687:7687"
    volumes:
      - neo4jdata:/data
  pgdata:
  redisdata:
  neo4jdata:

```

7.2 Kubernetes Deployment Overview

Namespace	Deployments	HPA Rule	Notes
aaes-core	aaes-api, aaes-worker (x4 replicas), aaes-ui	CPU > 70% → scale to 8 replicas	Core system pods. Priority class: high.
aaes-infra	postgres (StatefulSet), redis (StatefulSet), neo4j (StatefulSet)	No HPA — StatefulSet with PVC	Persistent storage. Backup via Velero daily.
aaes-venture	module9-14 workers (when ENABLE_VENTURE_MODULES=true)	Queue depth > 50 → scale to 4 replicas	Disabled by default. Activated Phase 5+.
aaes-monitoring	prometheus, grafana, alertmanager	No HPA	Single replica. Alert rules in alertmanager config.

Ingress Configuration: NGINX Ingress Controller with TLS termination via AWS ACM. Routing rules:

- `app.aaes-os.com` → `aaes-ui:3000`
- `api.aaes-os.com` → `aaes-api:4000`
- `metrics.aaes-os.com` → `grafana:3001` (VPN-only)

7.3 AWS Infrastructure

AWS Service	Configuration	Purpose
EKS	v1.30, managed node groups, t3.xlarge workers	Primary Kubernetes cluster for all services
RDS PostgreSQL	db.r6g.large, Multi-AZ, automated backups 7-day	Production PostgreSQL database
ElastiCache Redis	cache.r7g.medium, cluster mode enabled	Managed Redis for sessions, cache, queues
S3	Versioning enabled, SSE-S3 encryption, lifecycle	Binary asset storage (images, documents)

AWS Service	Configuration	Purpose
	90-day archive	
CloudFront	Origin: S3 + ALB. Cache: 24h for assets, no-cache for API	CDN for UI static assets and S3 asset delivery
Route 53	Hosted zone for aaes-os.com, latency-based routing	DNS management and routing
ACM	Wildcard cert: *.aaes-os.com, auto-renewing	TLS certificate management
ECR	Private registry per service, image scan on push	Docker image storage and versioning
Secrets Manager	Rotation: 90 days for DB passwords, 30 days for API keys	Secure storage for all credentials
CloudWatch	Log groups per service, 30-day retention, metric alarms	Centralized logging and alerting

8. Testing Strategy

8.1 Test Coverage Requirements

Layer	Unit Tests (Files)	Integration Tests	E2E Tests	Coverage Target
Layer 0 — PCL	test_pcl.py, test_shorthand_codes.py, test_genome_format.py	Compile → Store flow (PCL → PostgreSQL)	Full prompt string → stored genome	90%
Layer 1 — Alpha	Each pipeline stage class in isolation	Alpha full pipeline (genome → asset)	Intent → generated asset → Zeta stored	85%
Layer 2 — Beta	Each governance agent	Council review flow (4 agents → decision)	Alpha output → Beta gate → Zeta approved	90%

Layer	Unit Tests (Files)	Integration Tests	E2E Tests	Coverage Target
	independently		asset	
Layer 3 — Gamma	test_gamma_core.py, test_pattern_engine.py	Event ingestion → knowledge node creation	Full observe-abstract-model-apply loop	80%
Layer 4 — Delta	test_delta_engine.py, test_scenario_tester.py	Scenario → simulation → result	Hypothesis → simulation → Epsilon workflow	80%
Layer 5 — Epsilon	test_epsilon_executor.py, test_workflow_builder.py	Workflow build → step execution	Simulation result → workflow → action executed	80%
Layer 6 — Zeta	test_zeta_store.py, test_lineage_tracker.py	Store → retrieve → lineage chain	Full asset provenance chain verification	85%
Layer 7 — Omega	test_omega_coordinator.py, test_alignment_check.py	Conflict arbitration flow	Cross-layer conflict → resolution → audit log	80%
Layer 8 — Genesis	test_genesis.py, test_identity_init.py	Bootstrap sequence, idempotency test	Cold-start → fully operational system state	85%
API Surface	Route handler unit tests (Jest)	Middleware chain (auth → rateLimit → governance)	Full HTTP request cycle per endpoint	85%

8.2 Test Commands

```
# ----- # Python
Core Engine Tests (pytest) #
----- # Run all
Python tests with coverage pytest core/ --cov=core --cov-report=term-missing
--cov-report=html # Run a specific layer's tests pytest
core/layer0_prompt_engine/tests/ -v pytest core/layer1_alpha/tests/ -v pytest
core/layer2_beta/tests/ -v # Run with verbose output and stop on first
failure pytest core/ -v -x # Run only tests marked as unit (fast) pytest
core/ -m unit # Run only integration tests pytest core/ -m integration #
----- # Python
Linting and Type Checking #
```

```

----- # Ruff
(linter + formatter) ruff check core/ ruff format core/ # Mypy (type
checking) mypy core/ --strict --ignore-missing-imports #
----- # TypeScript
API Tests (Jest) #
----- # Run all
API tests npm test # Run with coverage npm run test:coverage # Run in watch
mode (development) npm run test:watch # Run a specific test file npx jest
api/routes/prompt.routes.test.ts #
----- # TypeScript
Linting and Type Checking #
----- # ESLint
npm run lint # TypeScript compiler check npx tsc --noEmit #
----- # E2E Tests
(Playwright) #
----- # Run full
E2E suite npm run test:e2e # Run E2E in headless mode npx playwright test --
reporter=html #
----- # Database
Migration Tests #
----- # Apply
schema to test database PGPASSWORD=password psql -U aaes_user -d aaes_os_test
-f db/schema.sql # Run migration scripts npm run migrate:test

```

9. Build Phase Implementation Checklist

The following phased checklist governs the build sequence for AAES-OS. Each phase must reach its target coverage and pass all integration tests before the next phase begins. Phases are sequential; items within a phase may be parallelized.

Phase 1 — Prompt Engine (Layer 0) — Target: Week 1-2

```

[ ] Implement ShorthandRegistry with all 6 initial codes (FPS, HUD, ARCH,
SYS, ENV, URB) [ ] Build PCL compile() method with regex tokenizer [ ] Create
PromptGenome dataclass + JSON serializer/deserializer [ ] Implement
GenomeLibrary with PostgreSQL persistence [ ] Populate genome_library.json
with 10 starter genomes [ ] Write unit tests for all 6 shorthand code
expansions (target: 90% coverage) [ ] Write integration test: compile() →
PostgreSQL store → retrieve [ ] Create POST /api/v1/prompt/compile endpoint [
] Create GET /api/v1/prompt/genome/:id endpoint [ ] Build minimal UI: prompt
input field + compiled genome output viewer [ ] Document all shorthand codes
in README

```

Phase 2 — AAES Alpha Creative Pipeline (Layer 1) — Target: Week 3-5

```

[ ] Implement IntentParser with initial intent taxonomy (generate_image,
generate_document, etc.) [ ] Build Generator class with OpenAI API

```

integration (text generation) [] Build Generator integration with Replicate API (image generation) [] Implement Evaluator with full rubric schema (5 criteria) [] Connect MemoryWriter to Zeta store (PostgreSQL + S3) [] Build Improver loop (max 3 iterations, re-evaluate after each) [] Implement PipelineContext and failure handling [] Set up Alpha pipeline full E2E test (genome → asset → Zeta stored) [] Create POST /api/v1/alpha/generate endpoint with async job support [] Create GET /api/v1/alpha/asset/:id endpoint with presigned S3 URL [] Add Alpha pipeline progress tracking via Redis [] Build UI: asset viewer component with quality score display

Phase 3 — Commercial Product Launch — Target: Week 6-7

[] Package top-scoring starter genomes into Starter Kit v1 product (JSON + PDF) [] Build genome export CLI tool (export to Notion-compatible markdown) [] Set up Gumroad product page for Genome Starter Kit (\$49 price point) [] Integrate Stripe for direct payment option [] Create usage analytics dashboard (genomes created, assets generated, quality distribution) [] Build purchasable export: curated genome library as PDF reference guide [] Set up email capture and delivery workflow for digital product [] Write product landing page copy and publish

Phase 4 — Consulting CRM and FOS Dashboard — Target: Week 8-9

[] Build FOS Dashboard: priorities view, decisions tracker, active projects [] Create client onboarding workflow template (Epsilon-powered) [] Set up consulting CRM module (contacts, engagements, deliverables) [] Build project status views linked to Epsilon workflow engine [] Create client deliverable packaging flow (genome → asset → export) [] Add client portal access with scoped JWT tokens [] Integrate calendar and milestone tracking into project views

Phase 5 — Full Layer Expansion — Target: Week 10-16

[] Implement Beta agent council (all 4 agents: Vision, Systems, Culture, Innovation) [] Implement IdentityGuard in beta_council.py [] Wire governance middleware: all Alpha routes auto-routed through Beta [] Implement Gamma knowledge ingestor with external document pipeline [] Implement Gamma pattern_engine.py (observe → abstract loop) [] Implement Delta scenario_tester.py (Monte Carlo simulation, 1000 runs) [] Implement Delta digital_twin.py (state snapshot + replay) [] Implement Epsilon workflow_builder.py (scenario result → workflow) [] Implement Epsilon epsilon_executor.py (step-by-step action execution) [] Wire all layers through Omega coordinator (orchestrate + arbitrate) [] Implement Omega alignment_check.py (alignment score gate) [] Run Genesis bootstrap sequence end-to-end in staging environment [] Enable ENABLE_VENTURE_MODULES flag; activate Modules 9-14 [] Run full system E2E: Genesis → prompt → genome → asset → Beta → Zeta → Epsilon action [] Deploy to EKS production cluster via GitHub Actions CI/CD pipeline [] Configure Prometheus alerts and Grafana dashboards

10. Appendices

A. Glossary of Code Terms

Term	Definition
AAES-OS	Adaptive AI Execution System — Operating System. The full multi-layer AI platform defined by this document.
PCL	Prompt Compiler Layer. Layer 0. Converts shorthand codes and base text into structured PromptGenome objects.
PromptGenome	The core data structure produced by Layer 0. A fully expanded, scored, and versioned specification of a creative intent, analogous to a DNA sequence for generative outputs.
ShorthandCode	A bracketed tag (e.g., [HUD], [ARCH]) that expands to a multi-dimensional style and context specification during PCL compilation.
Alpha	Layer 1. The Creative Organism. Executes the six-stage generative pipeline from genome to evaluated asset.
Beta	Layer 2. Governance Intelligence. Four-agent council that reviews and approves or rejects all Alpha-generated assets.
Gamma	Layer 3. Adaptive Intelligence. The learning loop that observes, abstracts, models, applies, and validates system patterns.
Delta	Layer 4. Research + Simulation. Probabilistic scenario modeling and digital twin construction for safe foresight.
Epsilon	Layer 5. Action Layer. The only layer authorized to execute real-world side effects (API calls, file writes, deployments).
Zeta	Layer 6. Stewardship. Long-term

Term	Definition
	memory, provenance tracking, knowledge graph, and semantic retrieval.
Omega	Layer 7. Coordination. System-wide arbiter, conflict resolver, and alignment verifier. Last gate before Epsilon actions.
Genesis	Layer 8. Bootstrap protocol. Establishes identity, loads mission, and deploys all agents in a strictly ordered sequence.
Fitness Score	A 0-10 numerical rating assigned to a PromptGenome by the PCL, reflecting the richness and coherence of its shorthand composition.
Quality Score	A 0-10 weighted average produced by the Alpha Evaluator and confirmed by the Beta council, measuring the merit of a generated asset.
Alignment Score	A 0-10 score computed by Omega's alignment_check against core system values. Must exceed 7.0 for Epsilon authorization.
ProvenanceChain	The full ordered ancestry of an artifact from its origin genome through all derivative versions, maintained by Zeta.
FOS	Founder Operating System. Module 13. Dashboard and toolset for founder-level decision making, prioritization, and project coordination.
CBF	Civilization Builder Framework. Module 9. Long-horizon planning and systems design framework for large-scale civilizational projects.
VSF	Venture Studio Framework. Module 14. End-to-end workflow for launching, operating, and scaling new venture studio entities.

B. Architecture Decision Records (ADRs)

ADR-001: Python for Core Engine, TypeScript for API Surface

Status: Accepted | Date: 2026-06-01 | Author: Jon Halstead

Context: The system requires both AI/ML heavy processing (layer pipelines, learning loops) and a high-throughput HTTP API with strong typing.

Decision: Use Python 3.12 for all core layer logic (Layers 0–8) to leverage the Python AI/ML ecosystem (OpenAI SDK, SQLAlchemy, Celery, NumPy). Use TypeScript + Node.js for the API server to ensure type-safe request/response contracts and benefit from the npm ecosystem for Express middleware.

Consequence: Two runtime environments must be maintained. Inter-process communication between API and core engine occurs via Celery task queue (Redis broker), not direct function calls.

ADR-002: PostgreSQL as Primary Store with JSONB for Flexible Schemas

Status: Accepted | Date: 2026-06-01 | Author: Jon Halstead

Context: Several entities (agent_verdicts, steps, branches, connections) have variable structure that is difficult to normalize without many joins. A pure document store (MongoDB) lacks the transactional guarantees needed for governance decisions.

Decision: Use PostgreSQL 16 as the primary database. Structured fields use typed columns. Variable-structure fields use `JSONB` with GIN indexes for efficient querying.

Consequence: Schema migrations are required for structural changes but are managed via versioned migration scripts. `JSONB` fields are not intrinsically typed; application-layer validation is required.

ADR-003: Synchronous Governance Gate for Alpha Routes

Status: Accepted | Date: 2026-06-08 | Author: Jon Halstead

Context: Assets generated by Alpha must not be accessible to callers until Beta has reviewed them. Async governance (webhook) creates a window where unapproved assets could be accessed.

Decision: Governance middleware on `/alpha/` routes operates synchronously: the HTTP response is held until the Beta council returns a decision. Async mode

(webhook) is available as an opt-in feature flag for high-throughput enterprise use cases.

Consequence: `POST /api/v1/alpha/generate` latency includes the full Beta review time (target: < 5 seconds for standard assets). Clients must use appropriate HTTP timeout settings (minimum 30 seconds recommended).

ADR-004: Neo4j for Knowledge Graph, Pinecone for Vector Search

Status: Accepted | Date: 2026-06-10 | Author: Jon Halstead

Context: Zeta requires two distinct retrieval modalities: graph traversal for provenance and knowledge connections (Neo4j) and semantic similarity search for asset discovery (vector embeddings).

Decision: Deploy Neo4j for knowledge graph operations and Gamma pattern connections. Deploy Pinecone (managed) for vector embedding storage and similarity search. pgvector serves as a fallback for development environments without Pinecone access.

Consequence: Three separate data stores must be kept in sync for the Zeta layer. Zeta's `store_asset()` method is responsible for writing to all three atomically (using a saga pattern with compensating transactions on failure).

C. Known Technical Debt and Future Work

ID	Area	Issue	Priority	Target Phase
TD-001	Layer 1 — Alpha	Improver loop currently uses fixed max-3-iteration limit without adaptive stopping criteria. Should evaluate diminishing returns on	Medium	Phase 5

ID	Area	Issue	Priority	Target Phase
		quality delta between iterations.		
TD-002	Layer 3 — Gamma	Pattern engine uses heuristic frequency counting. Full ML-based pattern abstraction (clustering, embedding similarity) deferred pending training data accumulation.	High	Phase 5+
TD-003	Layer 6 — Zeta	Saga pattern for tri-store writes (PostgreSQL + Neo4j + Pinecone) in <code>store_asset()</code> requires compensating transaction implementation. Currently uses best-effort write with manual reconciliation.	High	Phase 5
TD-004	Layer 0 — PCL	Shorthand code registry is currently static JSON. Future: dynamic code registration via admin API to allow community-contributed codes without code deployment.	Low	Phase 6

ID	Area	Issue	Priority	Target Phase
TD-005	API	No GraphQL surface implemented yet. REST-only for Phase 1-3. GraphQL with DataLoader planned to reduce round-trips for complex UI queries (e.g., asset + genome + lineage in one request).	Medium	Phase 5
TD-006	Layer 4 — Delta	Digital twin currently creates static state snapshots. Live-replay and state diffing capabilities (comparing twin vs. real system divergence over time) deferred.	Low	Phase 6
TD-007	Infra	No multi-region deployment. Single us-east-1 region. Disaster recovery relies on RDS automated backups and S3 cross-region replication only. Active-active multi-region is Phase 7+	Medium	Phase 7

ID	Area	Issue	Priority	Target Phase
		scope.		
TD-008	Layer 3 — Gamma	Federated learning (network_learner.py) is a stub. Requires privacy-preserving federated averaging implementation and partner node protocol definition before activation.	Low	Phase 6

D. Third-Party License Inventory

Library / Service	License	Usage	Compliance Notes
Python (CPython)	PSF License	Core runtime	Permissive. No restrictions on commercial use.
SQLAlchemy	MIT	Python ORM	Permissive. Attribution required in NOTICE file.
Celery	BSD 3-Clause	Task queue	Permissive. Attribution required.
OpenAI Python SDK	MIT	AI generation + embeddings	OpenAI usage governed by OpenAI API Terms of Service. Generated content ownership per ToS.
Express.js	MIT	API server framework	Permissive. Attribution required.

Library / Service	License	Usage	Compliance Notes
Prisma	Apache 2.0	TypeScript ORM	Permissive. Patent grant included.
Next.js	MIT	Frontend framework	Permissive. Vercel attribution not required for self-hosted.
PostgreSQL	PostgreSQL License	Primary database	Permissive BSD-style. No restrictions on commercial use.
Redis	RSALv2 / SSPL v1 (Redis 7.4+)	Cache and queue	Redis 7.2 (pre-license-change) used. Pin version in Dockerfile. Review before upgrading.
Neo4j Community	GPL v3	Knowledge graph	Community edition is GPL. If embedded in distributed product, GPL applies. Neo4j Enterprise license required for SaaS. Evaluate before commercial distribution.
Pinecone	SaaS — Proprietary ToS	Vector store	Usage governed by Pinecone ToS. Data residency in US-East-1.
Stable Diffusion (Replicate)	CreativeML Open RAIL-M	Image generation	Commercial use permitted with restrictions. Output images may not be used to train competing models. Attribution recommended.
Auth0	SaaS — Proprietary ToS	Authentication	Usage governed by Auth0 ToS. Free tier up to 7,500 MAUs.

Library / Service	License	Usage	Compliance Notes
			Enterprise contract required at scale.
Prometheus	Apache 2.0	Metrics	Permissive. Patent grant included.
Grafana	AGPL v3 (OSS) / Commercial	Dashboards	OSS version used internally. AGPL requires source disclosure if Grafana is offered as a service to external users. Evaluate for client-facing deployments.

LEGAL ACTION REQUIRED

Neo4j Community Edition (GPL v3) and Grafana OSS (AGPL v3) have copyleft licenses that may impose obligations when bundled in commercial SaaS offerings. Legal review is required before AAES-OS is offered as a multi-tenant platform to external customers. Consider Neo4j Enterprise license and Grafana Cloud (commercial tier) for production SaaS deployments.

AAES-OS Code Breakdown — Technical Implementation Reference v1.0

Author: Jon Halstead | Generated: Tuesday, June 16, 2026, 12:01 PM EDT | Grayling Charter Township, MI

Classification: Internal Technical Reference — Confidential | All rights reserved, Jon Halstead 2026